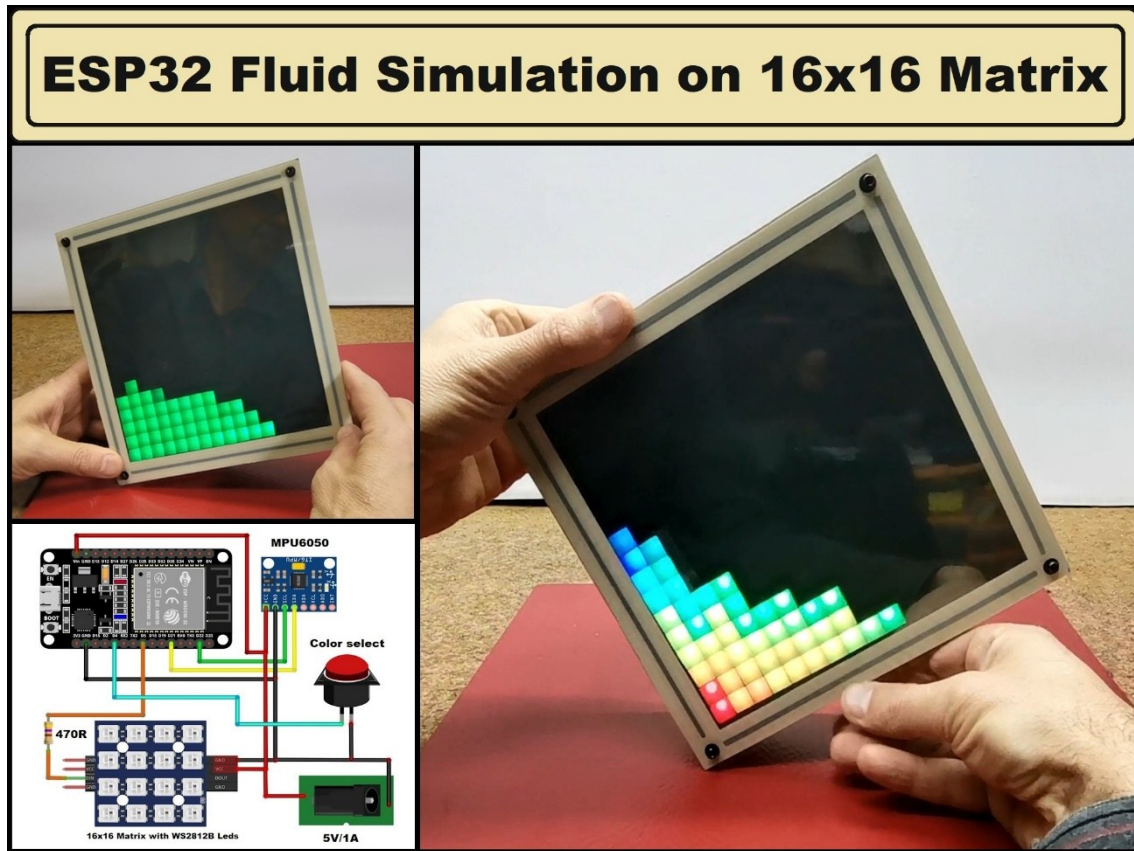


ESP32 Fluid simulation on 16x16 Led Matrix

This is a simple visually interesting toy for describing fluid dynamics.

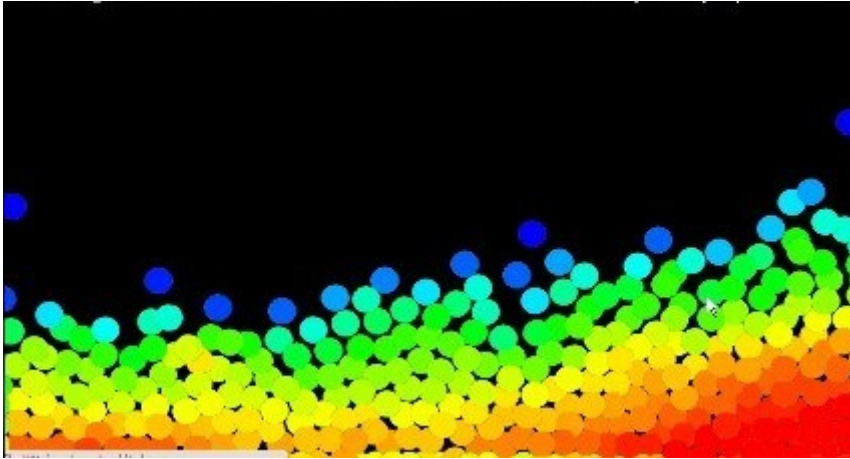


Components and supplies

- 1 LED Matrix Panel
- 1 MPU6050 Accelerometer
- 1 ESP32
- 1 Push Button

Project description

Fluid simulation is a way of replicating the movement and behavior of liquids and gases in different environments. It's widely used in fields like gaming, animation, engineering, and physics to create realistic visual effects and solve complex fluid-related problems.

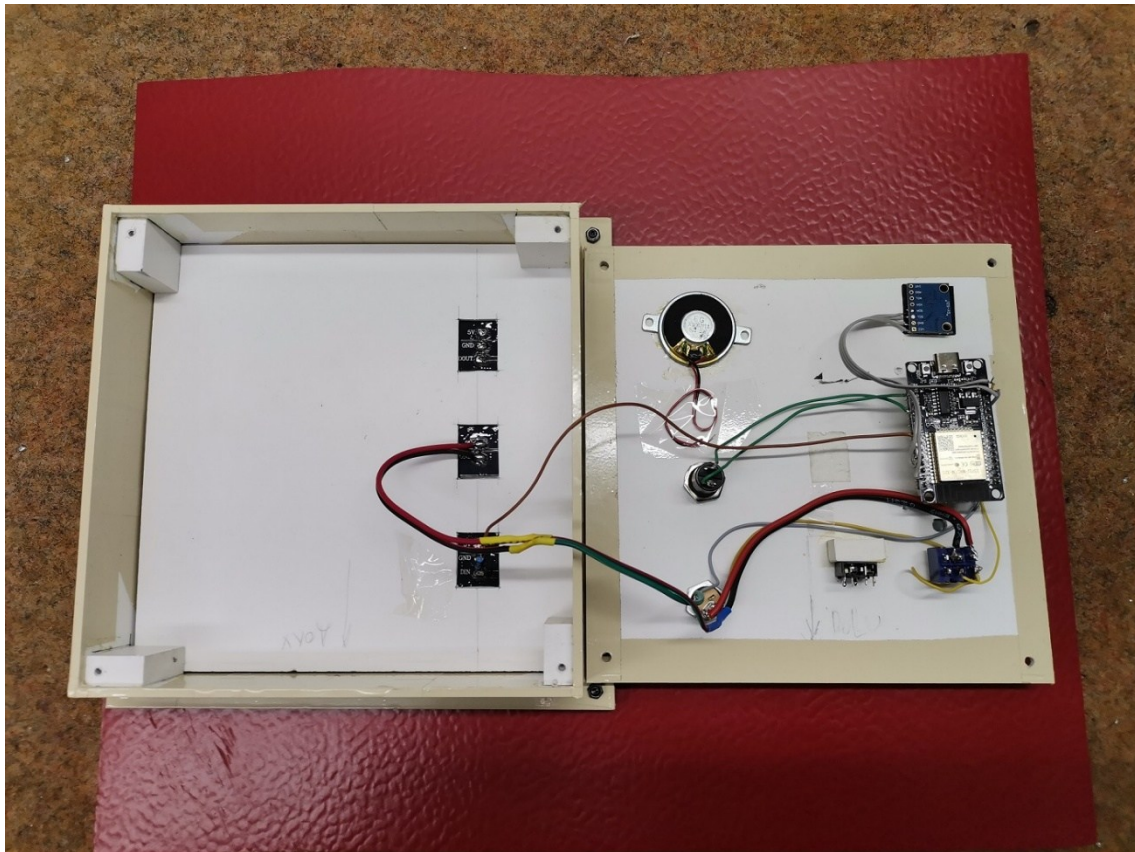


This time I will present you a very simple way to make a fluid motion simulator using a few components. This is a simulator with a relatively low resolution of 256 dots and for that purpose a Display made of 16x16 LEDs with WS2812B LED chips is used.

Specifically, I am using a cheap ready-made module with 16x16 LEDs. However, on this small "Display" I will create some really cool visualizations.

The device is extremely simple to build and consists of only a few components.

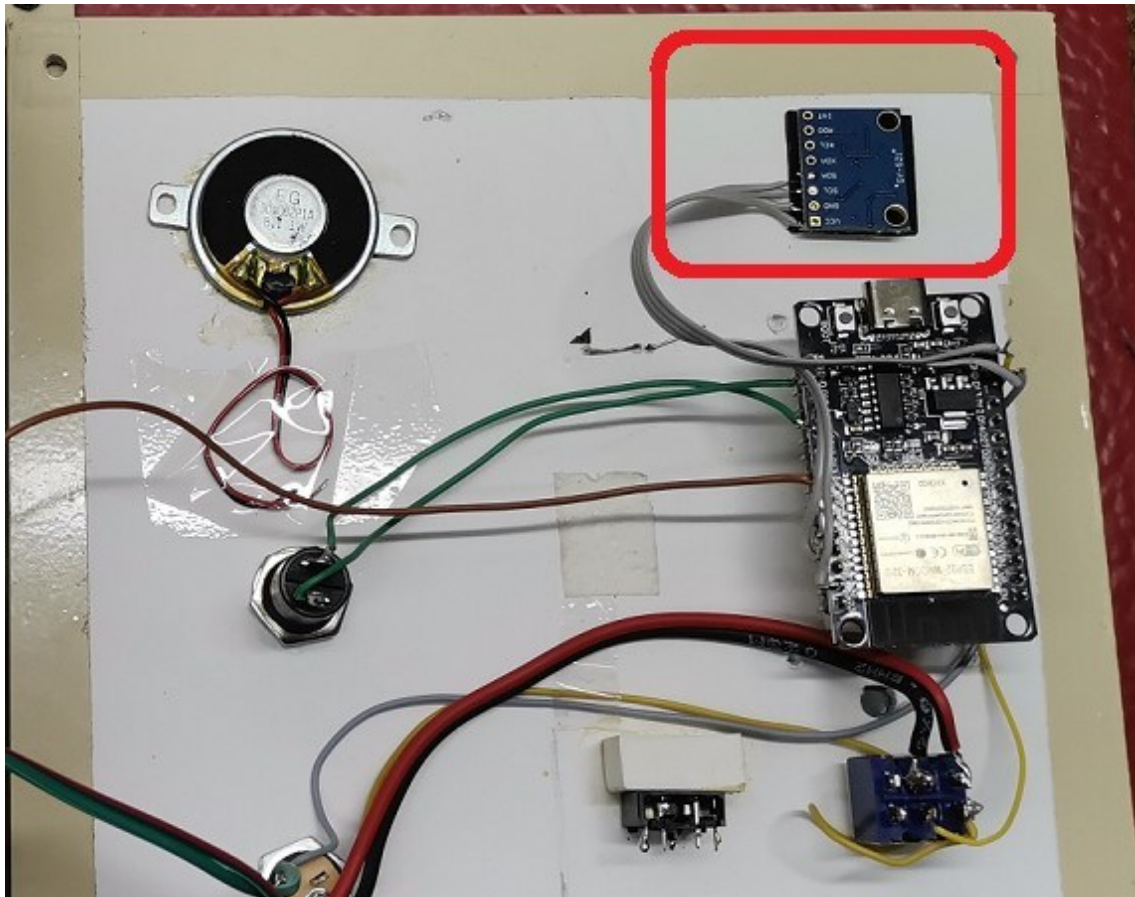
- ESP32 Microcontroller Dev Board
- MPU6050 accelerometer module
- 16x16 Led module with WS2812B chips
- and Button



This project is sponsored by [\\$ PCBWay \\$](#). They has all the services you need to create your project at the best price, whether is a scool project, or complex professional project. On [\\$ PCBWay \\$](#) you can share your experiences, or get inspiration for your next project. They also provide completed Surface mount SMT PCB assembly service at a best price, and ISO9001 quality control. Visit [\\$ pcbway.com \\$](#) for more services.

PCBWay

For this project I am using a box from one of my previous devices, for which I have also made a 3D printed grille for a better visual impression. Otherwise, even without this addition, the visual effect is impressive. It is important to note that the IMU sensor should be mounted in the way you see in the description, because otherwise you will get an undefined movement that does not comply with the laws of physics.



Now a few words about the software. The code is designed in a way that allows us to change multiple parameters, so we can simulate the movement of sand particles, liquids, gases, and other fluids.

First of all, we can change the number of active fluid particles and the light intensity of the LEDs. With the button, we can also choose one of the three colors for the LEDs that we have defined previously. At the beginning of the code, numerical values are given for some of the colors.


```
sketch_feb17a | Arduino 1.8.16
File Edit Sketch Tools Help

sketch_feb17a $
// Button for color switching

#define NUM_LEDS 256
#define MATRIX_WIDTH 16
#define MATRIX_HEIGHT 16
#define FLUID_PARTICLES 64 //80/64
#define BRIGHTNESS 30
#define NUM_COLORS 3 // Number of color options

// Structures
struct Vector2D {
    float x;
    float y;
};

struct Particle {
    Vector2D position;
    Vector2D velocity;
};

// Global variables
CRGB leds[NUM_LEDS];
MPU6050 mpu;
Particle particles[FLUID_PARTICLES];
Vector2D acceleration = {0, 0};

// Color switching variables
uint8_t currentColorIndex = 0;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 200;

// Define the colors (you can change these hue values)
const uint8_t COLORS[NUM_COLORS] = {
    160, // Blue
    0, // Red
    96 // Green
};

// Mutex for synchronization
portMUX_TYPE dataMux = portMUX_INITIALIZER_UNLOCKED;

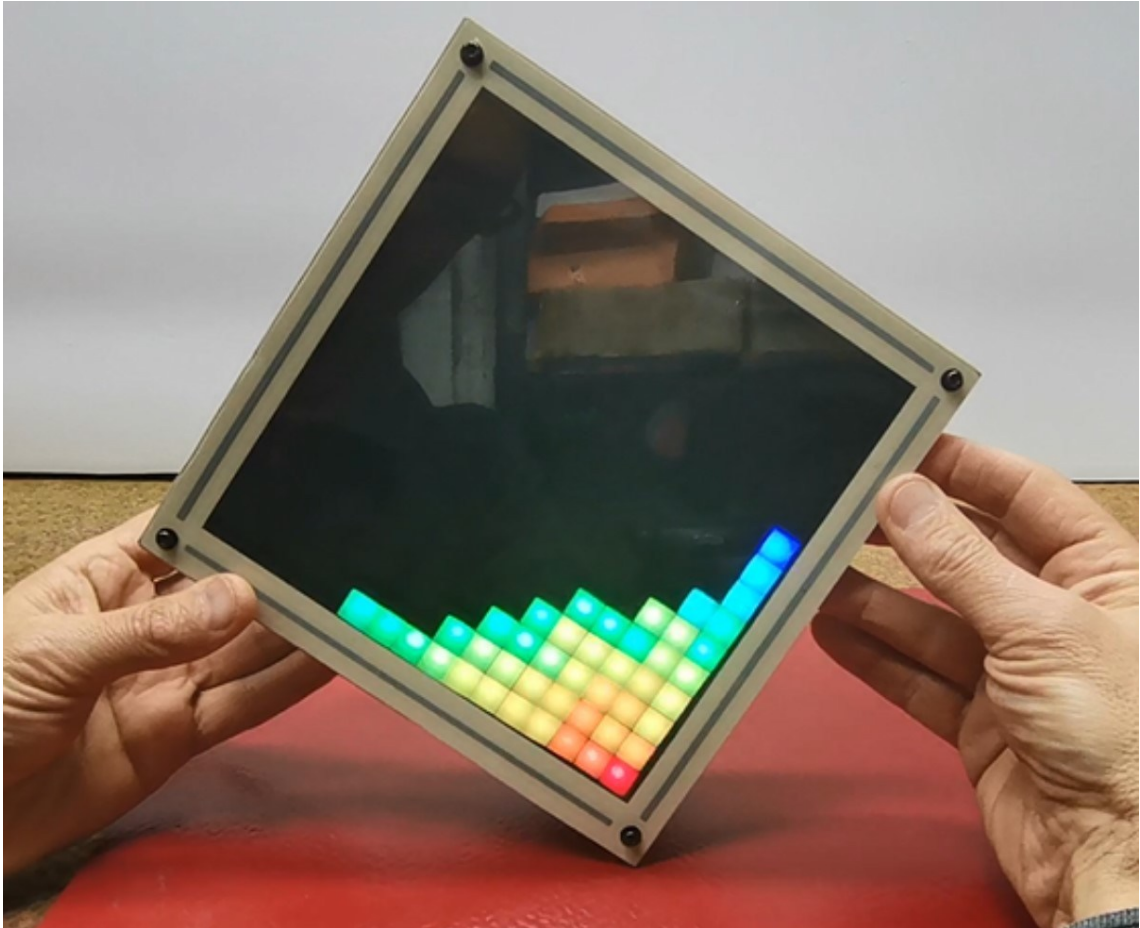
// Constants for physics
const float GRAVITY = 0.08f; //0.3f /098f
const float DAMPING = 0.92f; //0.99f /0.9f
const float MAX_VELOCITY = 0.6f; //0.6f /2.9f

// Function prototypes
void setup();
void loop();

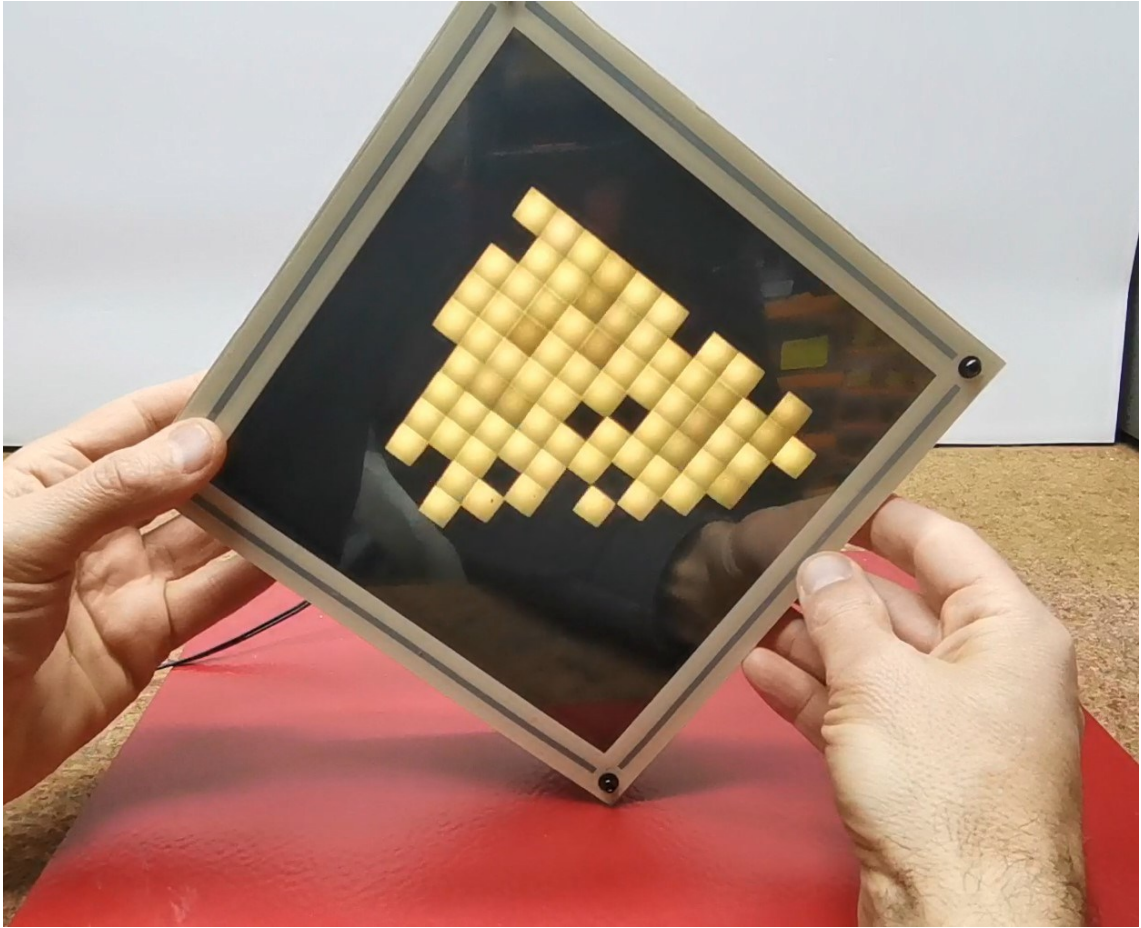
// Disabled, Default 4MB with spiffs (1.2MB APP/1.5MB SPIFFS), 240MHz (WiFi/BT), QIO, 80MHz, 4MB (32Mb), 921600, Core 1, Core 1, None, Disabled, Disabled on COM4
```

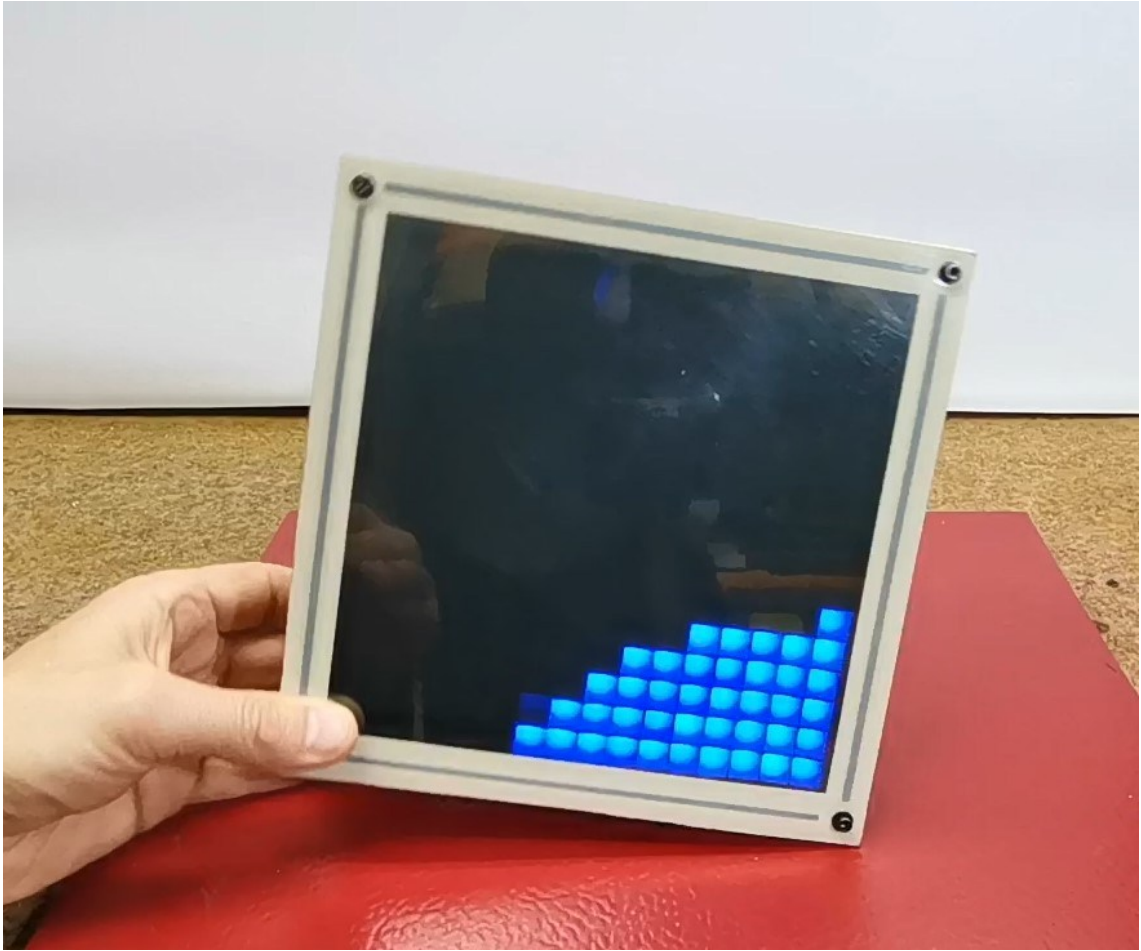
I will also present you a version of the code where the color of the particles changes dynamically depending on their location, which gives an even more interesting visual effect.

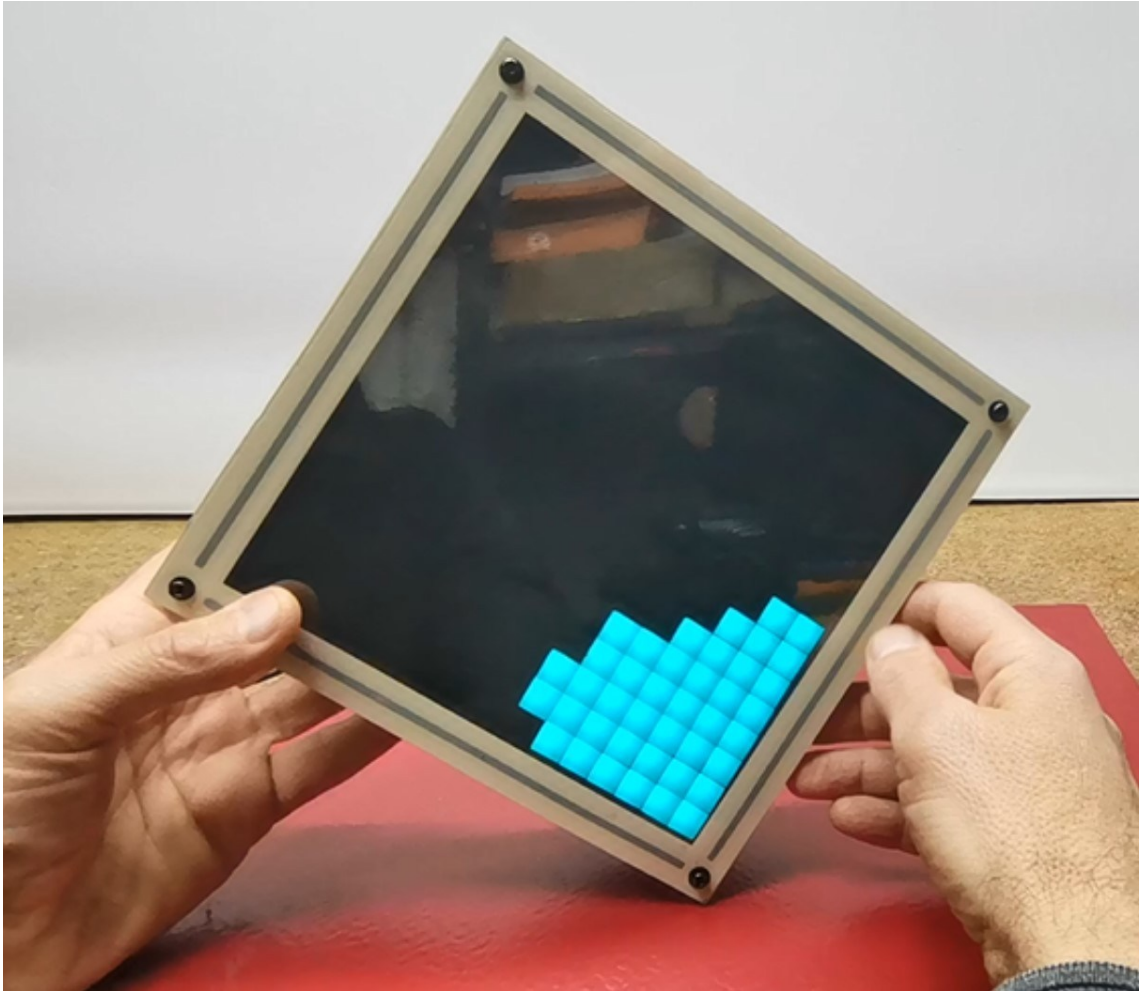
Then follow the basic physical quantities in the form of constants. By combining their values, various ways of moving fluids are obtained.

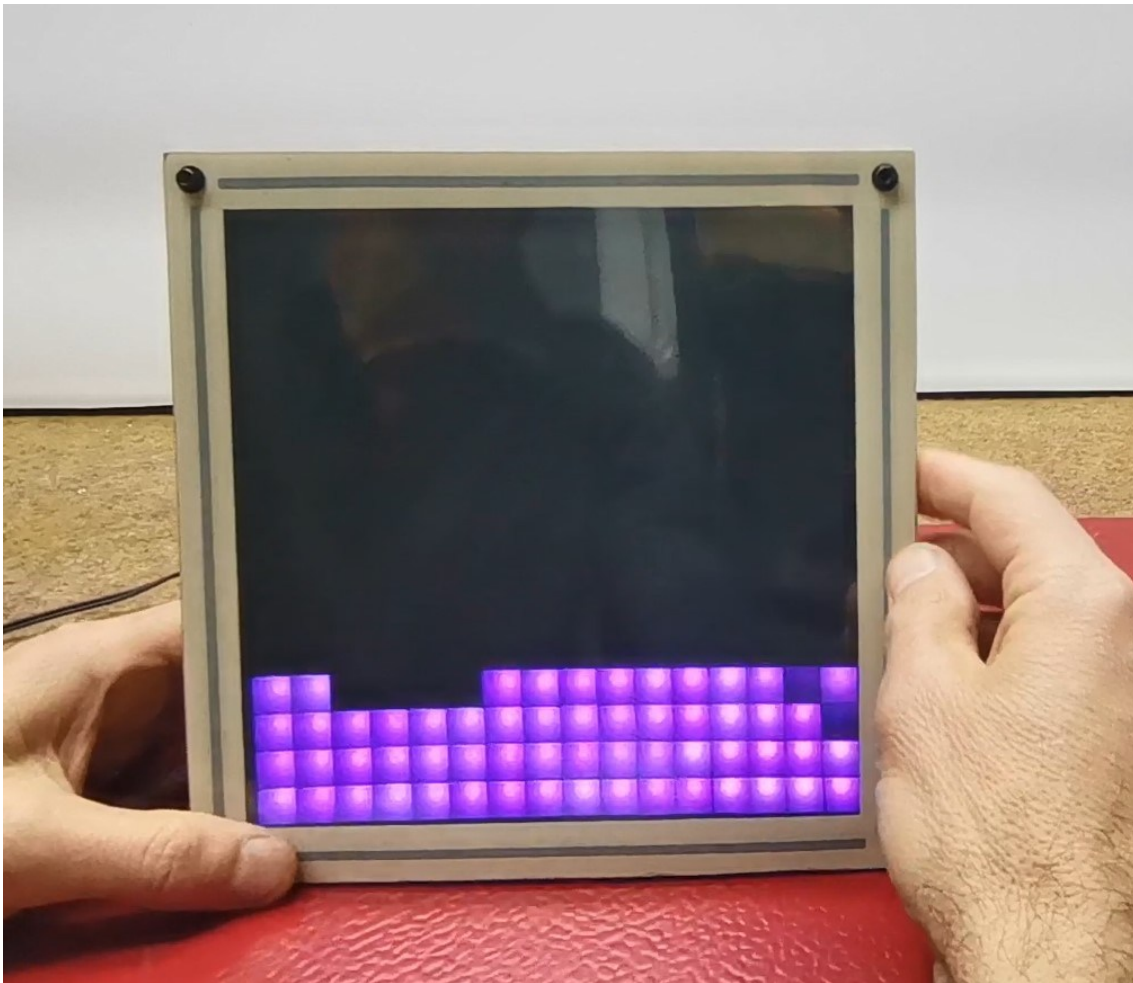


Now let's see how the device behaves in real conditions. I'll present you with just a few different situations, and you can experiment with many different combinations of physical constants.









And finally, a brief conclusion. This simple device serves only as a visual presentation of the way several different fluids move, i.e. primarily as a visually interesting toy for describing fluid dynamics.

CODE Color palette

cpp

```
#include <FastLED.h>
#include <Wire.h>
#include <MPU6050.h>

// Pin definitions
#define LED_PIN 5
#define SDA_PIN 21
#define SCL_PIN 22
```

```
#define NUM_LEDS      256
#define MATRIX_WIDTH  16
#define MATRIX_HEIGHT 16
#define FLUID_PARTICLES 64
#define BRIGHTNESS    100

// Structures
struct Vector2D {
    float x;
    float y;
};

struct Particle {
    Vector2D position;
    Vector2D velocity;
};

// Global variables
CRGB leds[NUM_LEDS];
MPU6050 mpu;
Particle particles[FLUID_PARTICLES];
Vector2D acceleration = {0, 0};

// Mutex for synchronization
portMUX_TYPE dataMux =
portMUX_INITIALIZER_UNLOCKED;

// Adjusted constants for smoother motion
const float GRAVITY = 0.3f;    //0.03f
const float DAMPING = 0.60f;  // 0.98f
const float MAX_VELOCITY = 0.7f; //0.3f
const float MIN_MOVEMENT = 0.001f; //0.001f
```

```

// Function prototypes
void initMPU6050();
void initLEDs();
void initParticles();
void updateParticles();
void drawParticles();
void MPU6050Task(void *parameter);
void LEDTask(void *parameter);

// Function to convert x,y coordinates to LED index
int xy(int x, int y) {
    x = constrain(x, 0, MATRIX_WIDTH - 1);
    y = constrain(y, 0, MATRIX_HEIGHT - 1);
    return (y & 1) ? (y * MATRIX_WIDTH +
(MATRIX_WIDTH - 1 - x)) : (y * MATRIX_WIDTH + x);
}

void drawParticles() {
    FastLED.clear();

    // Create occupancy grid
    bool occupied[MATRIX_WIDTH][MATRIX_HEIGHT] =
{{false}};

    // Get and smooth gravity direction
    static Vector2D lastGravityDir = {0, 1};
    Vector2D currentGravityDir;

    portENTER_CRITICAL(&dataMux);
    currentGravityDir = acceleration;
    portEXIT_CRITICAL(&dataMux);

    const float GRAVITY_SMOOTHING = 0.95f;

```



```

    lastGravityDir.x = lastGravityDir.x *
GRAVITY_SMOOTHING + currentGravityDir.x * (1 -
GRAVITY_SMOOTHING);
    lastGravityDir.y = lastGravityDir.y *
GRAVITY_SMOOTHING + currentGravityDir.y * (1 -
GRAVITY_SMOOTHING);

    // Normalize gravity vector
    float gravMagnitude = sqrt(lastGravityDir.x *
lastGravityDir.x + lastGravityDir.y *
lastGravityDir.y);
    Vector2D gravityDir = {0, 1}; // Default down
direction

    if (gravMagnitude > 0.1f) {
        gravityDir.x = lastGravityDir.x /
gravMagnitude;
        gravityDir.y = lastGravityDir.y /
gravMagnitude;
    }

    // Calculate heights and prepare for drawing
    float heights[FLUID_PARTICLES];
    float minHeight = 1000;
    float maxHeight = -1000;

    for (int i = 0; i < FLUID_PARTICLES; i++) {
        heights[i] = -(particles[i].position.x *
gravityDir.x +
                        particles[i].position.y *
gravityDir.y);
        minHeight = min(minHeight, heights[i]);
        maxHeight = max(maxHeight, heights[i]);
    }

```

```

    float heightRange = max(maxHeight - minHeight,
1.0f);

    // Draw particles
    int visibleCount = 0;
    for (int i = 0; i < FLUID_PARTICLES; i++) {
        int x =
round(constrain(particles[i].position.x, 0,
MATRIX_WIDTH - 1));
        int y =
round(constrain(particles[i].position.y, 0,
MATRIX_HEIGHT - 1));

        if (!occupied[x][y]) {
            int index = xy(x, y);
            if (index >= 0 && index < NUM_LEDS) {
                float relativeHeight = (heights[i]
- minHeight) / heightRange;
                uint8_t hue = relativeHeight *
160; // Map from 0 (red) to 160 (blue)
                uint8_t sat = 255; // Full
saturation for vibrant colors
                uint8_t val = 220 +
(relativeHeight * 35); // Slightly brighter at
top;

                leds[index] = CHSV(hue, sat, val);
                occupied[x][y] = true;
                visibleCount++;
            }
        } else {
            // Find nearest empty position
            for (int dx = -1; dx <= 1; dx++) {

```

```

        for (int dy = -1; dy <= 1; dy++) {
            if (dx == 0 && dy == 0)
continue;

            int newX = x + dx;
            int newY = y + dy;

            if (newX >= 0 && newX <
MATRIX_WIDTH &&
newY >= 0 && newY <
MATRIX_HEIGHT &&
!occupied[newX][newY]) {
                int index = xy(newX, newY);
                if (index >= 0 && index <
NUM_LEDS) {
                    float relativeHeight =
(heights[i] - minHeight) / heightRange;
                    uint8_t hue =
relativeHeight * 160; // Map from 0 (red) to 160
(blue)
                    uint8_t sat = 255; // Full
saturation for vibrant colors
                    uint8_t val = 220 +
(relativeHeight * 35); // Slightly brighter at top

                    leds[index] = CHSV(hue,
sat, val);

                    occupied[newX][newY] =
true;

                    visibleCount++;
                    goto particleDrawn;
                }
            }
        }
    }
}

```

```

        }
    }
    particleDrawn: continue;
}

// Debug output
static unsigned long lastDebugTime = 0;
if (millis() - lastDebugTime > 1000) {
    Serial.printf("Visible particles: %d of %d\n", visibleCount, FLUID_PARTICLES);
    lastDebugTime = millis();
}

FastLED.show();
}

```

```

void updateParticles() {
    Vector2D currentAccel;
    portENTER_CRITICAL(&dataMux);
    currentAccel = acceleration;
    portEXIT_CRITICAL(&dataMux);

    // Reduce acceleration sensitivity
    currentAccel.x *= 0.3f;
    currentAccel.y *= 0.3f;

    // Update and constrain each particle
    for (int i = 0; i < FLUID_PARTICLES; i++) {
        // Update velocity with acceleration
    }
}

```

```
        particles[i].velocity.x =
particles[i].velocity.x * 0.95f + (currentAccel.x *
GRAVITY);
        particles[i].velocity.y =
particles[i].velocity.y * 0.95f + (currentAccel.y *
GRAVITY);

        // Hard constrain velocity
        particles[i].velocity.x =
constrain(particles[i].velocity.x, -MAX_VELOCITY,
MAX_VELOCITY);
        particles[i].velocity.y =
constrain(particles[i].velocity.y, -MAX_VELOCITY,
MAX_VELOCITY);

        // Calculate new position
        float newX = particles[i].position.x +
particles[i].velocity.x;
        float newY = particles[i].position.y +
particles[i].velocity.y;

        // Strict boundary checking with bounce
        if (newX < 0.0f) {
            newX = 0.0f;
            particles[i].velocity.x =
fabs(particles[i].velocity.x) * DAMPING;
        }
        else if (newX >= (MATRIX_WIDTH - 1.0f)) {
            newX = MATRIX_WIDTH - 1.0f;
            particles[i].velocity.x = -
fabs(particles[i].velocity.x) * DAMPING;
        }

        if (newY < 0.0f) {
```



```

        newY = 0.0f;
        particles[i].velocity.y =
fabs(particles[i].velocity.y) * DAMPING;
    }
    else if (newY >= (MATRIX_HEIGHT - 1.0f)) {
        newY = MATRIX_HEIGHT - 1.0f;
        particles[i].velocity.y = -
fabs(particles[i].velocity.y) * DAMPING;
    }

    // Ensure positions are always within
bounds
    particles[i].position.x = constrain(newX,
0.0f, MATRIX_WIDTH - 1.0f);
    particles[i].position.y = constrain(newY,
0.0f, MATRIX_HEIGHT - 1.0f);

    // Additional safety check
    if (isnan(particles[i].position.x) ||
isnan(particles[i].position.y)) {
        particles[i].position.x = MATRIX_WIDTH
/ 2;
        particles[i].position.y = MATRIX_HEIGHT
/ 2;
        particles[i].velocity.x = 0;
        particles[i].velocity.y = 0;
    }
}

// Particle collision detection and resolution
for (int i = 0; i < FLUID_PARTICLES; i++) {
    for (int j = i + 1; j < FLUID_PARTICLES;
j++) {

```

```

        float dx = particles[j].position.x -
particles[i].position.x;
        float dy = particles[j].position.y -
particles[i].position.y;
        float distSquared = dx * dx + dy * dy;

        if (distSquared < 1.0f && distSquared >
0.0f) {
            float dist = sqrt(distSquared);
            float nx = dx / dist;
            float ny = dy / dist;

            // Push particles apart
            float pushDistance = (1.0f - dist)
* 0.5f;

            float pushX = nx * pushDistance;
            float pushY = ny * pushDistance;

            // Update positions while ensuring
they stay in bounds
            particles[i].position.x =
constrain(particles[i].position.x - pushX, 0.0f,
MATRIX_WIDTH - 1.0f);
            particles[i].position.y =
constrain(particles[i].position.y - pushY, 0.0f,
MATRIX_HEIGHT - 1.0f);
            particles[j].position.x =
constrain(particles[j].position.x + pushX, 0.0f,
MATRIX_WIDTH - 1.0f);
            particles[j].position.y =
constrain(particles[j].position.y + pushY, 0.0f,
MATRIX_HEIGHT - 1.0f);

            // Exchange velocities with damping

```

```

        float tempVelX =
particles[i].velocity.x;
        float tempVelY =
particles[i].velocity.y;
        particles[i].velocity.x =
particles[j].velocity.x * DAMPING;
        particles[i].velocity.y =
particles[j].velocity.y * DAMPING;
        particles[j].velocity.x = tempVelX
* DAMPING;
        particles[j].velocity.y = tempVelY
* DAMPING;
    }
}
}
}

```

```

void initMPU6050() {
    Serial.println("Initializing MPU6050...");
    mpu.initialize();

    if (!mpu.testConnection()) {
        Serial.println("MPU6050 connection
failed!");
        while (1) {
            delay(100);
        }
    }

    mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
    Serial.println("MPU6050 initialized");
}

```

```

}

void initLEDs() {
    Serial.println("Initializing LEDs...");
    FastLED.addLeds<WS2812B, LED_PIN, GRB>(leds,
NUM_LEDS);
    FastLED.setBrightness(BRIGHTNESS);
    FastLED.clear(true);
    Serial.println("LEDs initialized");
}

void initParticles() {
    Serial.println("Initializing particles...");
    int index = 0;

    for (int y = MATRIX_HEIGHT - 4; y <
MATRIX_HEIGHT; y++) {
        for (int x = 0; x < MATRIX_WIDTH && index <
FLUID_PARTICLES; x++) {
            Serial.printf("Initializing particle %d
at x=%d, y=%d\n", index, x, y);
            particles[index].position =
{static_cast<float>(x), static_cast<float>(y)};
            particles[index].velocity = {0.0f,
0.0f};
            index++;
        }
    }

    Serial.printf("Total particles initialized:
%d\n", index);
    Serial.println("Particles initialized");
}

```

```

void MPUTask(void *parameter) {
    while (true) {
        int16_t ax, ay, az;
        mpu.getAcceleration(&ax, &ay, &az);

        portENTER_CRITICAL(&dataMux);
        acceleration.x = -constrain(ax / 16384.0f,
-1.0f, 1.0f);
        acceleration.y = constrain(ay / 16384.0f, -
1.0f, 1.0f);
        portEXIT_CRITICAL(&dataMux);

        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

void LEDTask(void *parameter) {
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency =
pdMS_TO_TICKS(16);

    while (true) {
        updateParticles();
        drawParticles();
        vTaskDelayUntil(&xLastWakeTime,
xFrequency);
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("Starting initialization...");
}

```

```
Wire.begin(SDA_PIN, SCL_PIN);
Wire.setClock(400000);

initMPU6050();
initLEDs();
initParticles();

xTaskCreatePinnedToCore(
    MPU6050Task,
    "MPU6050Task",
    4096,
    NULL,
    2,
    NULL,
    0
);

xTaskCreatePinnedToCore(
    LEDTask,
    "LEDTask",
    4096,
    NULL,
    1,
    NULL,
    1
);

Serial.println("Setup complete");
}

void loop() {
    vTaskDelete(NULL);
}
```


CODE Colors Button

cpp

```
// colors:  0 = Red, 32 = Orange, 64 = Yellow, 96
= Green, 128 = Aqua, 160 = Blue, 192 = Purple, 224
= Pink

#include <FastLED.h>
#include <Wire.h>
#include <MPU6050.h>

// Pin definitions
#define LED_PIN      5
#define SDA_PIN      21
#define SCL_PIN      22
#define BUTTON_PIN  4    // Button for color
switching

#define NUM_LEDS      256
#define MATRIX_WIDTH  16
#define MATRIX_HEIGHT 16
#define FLUID_PARTICLES 64 //80/64
#define BRIGHTNESS    30
#define NUM_COLORS    3    // Number of color options

// Structures
struct Vector2D {
    float x;
    float y;
};

struct Particle {
    Vector2D position;
    Vector2D velocity;
```

```

};

// Global variables
CRGB leds[NUM_LEDS];
MPU6050 mpu;
Particle particles[FLUID_PARTICLES];
Vector2D acceleration = {0, 0};

// Color switching variables
uint8_t currentColorIndex = 0;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 200;

// Define the colors (you can change these hue
values)
const uint8_t COLORS[NUM_COLORS] = {
    160, // Blue
    0,   // Red
    96   // Green
};

// Mutex for synchronization
portMUX_TYPE dataMux =
portMUX_INITIALIZER_UNLOCKED;

// Constants for physics
const float GRAVITY = 0.08f; //0.3f /0.98f
const float DAMPING = 0.92f; //0.99f /0.9f
const float MAX_VELOCITY = 0.6f; //0.6f /2.9f

// Function prototypes
void initMPU6050();
void initLEDs();
void initParticles();

```

```

void updateParticles();
void drawParticles();
void MPUtask(void *parameter);
void LEDTask(void *parameter);
void checkButton();

// Function to convert x,y coordinates to LED index
int xy(int x, int y) {
    x = constrain(x, 0, MATRIX_WIDTH - 1);
    y = constrain(y, 0, MATRIX_HEIGHT - 1);
    return (y & 1) ? (y * MATRIX_WIDTH +
(MATRIX_WIDTH - 1 - x)) : (y * MATRIX_WIDTH + x);
}

void checkButton() {
    static bool lastButtonState = HIGH;
    bool buttonState = digitalRead(BUTTON_PIN);

    if (buttonState == LOW && lastButtonState ==
HIGH) { // Button pressed
        if ((millis() - lastDebounceTime) >
debounceDelay) {
            currentColorIndex = (currentColorIndex
+ 1) % NUM_COLORS;
            lastDebounceTime = millis();
        }
    }
    lastButtonState = buttonState;
}

void drawParticles() {
    FastLED.clear();

```

```

    bool occupied[MATRIX_WIDTH][MATRIX_HEIGHT] =
{{false}};

    struct ParticleIndex {
        int index;
        float position;
    };

    ParticleIndex sortedParticles[FLUID_PARTICLES];
    for (int i = 0; i < FLUID_PARTICLES; i++) {
        sortedParticles[i].index = i;
        sortedParticles[i].position =
particles[i].position.y * MATRIX_WIDTH +
particles[i].position.x;
    }

    for (int i = 0; i < FLUID_PARTICLES - 1; i++) {
        for (int j = 0; j < FLUID_PARTICLES - i -
1; j++) {
            if (sortedParticles[j].position >
sortedParticles[j + 1].position) {
                ParticleIndex temp =
sortedParticles[j];
                sortedParticles[j] =
sortedParticles[j + 1];
                sortedParticles[j + 1] = temp;
            }
        }
    }

    for (int i = 0; i < FLUID_PARTICLES; i++) {
        int particleIndex =
sortedParticles[i].index;

```

```

        int x =
round(particles[particleIndex].position.x);
        int y =
round(particles[particleIndex].position.y);

        x = constrain(x, 0, MATRIX_WIDTH - 1);
        y = constrain(y, 0, MATRIX_HEIGHT - 1);

        if (!occupied[x][y]) {
            int index = xy(x, y);
            if (index >= 0 && index < NUM_LEDS) {
                float speed = sqrt(
                    particles[particleIndex].velocity.x *
particles[particleIndex].velocity.x +
                    particles[particleIndex].velocity.y *
particles[particleIndex].velocity.y
                );

                uint8_t hue =
COLORS[currentColorIndex];
                uint8_t sat = 255;
                uint8_t val = constrain(180 +
(speed * 50), 180, 255);

                leds[index] = CHSV(hue, sat, val);
                occupied[x][y] = true;
            }
        } else {
            for (int r = 1; r < 3; r++) {
                for (int dx = -r; dx <= r; dx++) {
                    for (int dy = -r; dy <= r;
dy++) {
                        if (abs(dx) + abs(dy) == r)

```

```

int newX = x + dx;
int newY = y + dy;
if (newX >= 0 && newX <
MATRIX_WIDTH &&
newY >= 0 && newY <
MATRIX_HEIGHT &&
!occupied[newX][new
Y]) {
int index =
xy(newX, newY);
if (index >= 0 &&
index < NUM_LEDS) {
leds[index] =
CHSV(COLORS[currentColorIndex], 255, 180);
occupied[newX][
newY] = true;
goto
nextParticle;
}
}
}
}
}
}
}
nextParticle:
continue;
}
}
FastLED.show();
}

void updateParticles() {
Vector2D currentAccel;

```

```
portENTER_CRITICAL(&dataMux);
currentAccel = acceleration;
portEXIT_CRITICAL(&dataMux);

currentAccel.x *= 0.3f;
currentAccel.y *= 0.3f;

for (int i = 0; i < FLUID_PARTICLES; i++) {
    particles[i].velocity.x =
particles[i].velocity.x * 0.9f + (currentAccel.x *
GRAVITY);
    particles[i].velocity.y =
particles[i].velocity.y * 0.9f + (currentAccel.y *
GRAVITY);

    particles[i].velocity.x =
constrain(particles[i].velocity.x, -MAX_VELOCITY,
MAX_VELOCITY);
    particles[i].velocity.y =
constrain(particles[i].velocity.y, -MAX_VELOCITY,
MAX_VELOCITY);

    float newX = particles[i].position.x +
particles[i].velocity.x;
    float newY = particles[i].position.y +
particles[i].velocity.y;

    if (newX < 0.0f) {
        newX = 0.0f;
        particles[i].velocity.x =
fabs(particles[i].velocity.x) * DAMPING;
    }
    else if (newX >= (MATRIX_WIDTH - 1)) {
        newX = MATRIX_WIDTH - 1;
    }
}
```



```

        particles[i].velocity.x = -
fabs(particles[i].velocity.x) * DAMPING;
    }

    if (newY < 0.0f) {
        newY = 0.0f;
        particles[i].velocity.y =
fabs(particles[i].velocity.y) * DAMPING;
    }
    else if (newY >= (MATRIX_HEIGHT - 1)) {
        newY = MATRIX_HEIGHT - 1;
        particles[i].velocity.y = -
fabs(particles[i].velocity.y) * DAMPING;
    }

    particles[i].position.x = constrain(newX,
0.0f, MATRIX_WIDTH - 1);
    particles[i].position.y = constrain(newY,
0.0f, MATRIX_HEIGHT - 1);

    particles[i].velocity.x *= 0.95f;
    particles[i].velocity.y *= 0.95f;
}

for (int i = 0; i < FLUID_PARTICLES; i++) {
    for (int j = i + 1; j < FLUID_PARTICLES;
j++) {
        float dx = particles[j].position.x -
particles[i].position.x;
        float dy = particles[j].position.y -
particles[i].position.y;
        float distanceSquared = dx * dx + dy *
dy;

```

```

        if (distanceSquared < 1.0f) {
            float distance =
sqrt(distanceSquared);
            float angle = atan2(dy, dx);

            float repulsionX = cos(angle) *
0.5f;
            float repulsionY = sin(angle) *
0.5f;

            particles[i].position.x -=
repulsionX * 0.3f;
            particles[i].position.y -=
repulsionY * 0.3f;
            particles[j].position.x +=
repulsionX * 0.3f;
            particles[j].position.y +=
repulsionY * 0.3f;

            Vector2D avgVel = {
                (particles[i].velocity.x +
particles[j].velocity.x) * 0.5f,
                (particles[i].velocity.y +
particles[j].velocity.y) * 0.5f
            };

            particles[i].velocity = avgVel;
            particles[j].velocity = avgVel;
        }
    }
}

```

```

void initMPU6050() {

```

```

    Serial.println("Initializing MPU6050...");
    mpu.initialize();

    if (!mpu.testConnection()) {
        Serial.println("MPU6050 connection
failed!");
        while (1) {
            delay(100);
        }
    }

    mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);
    Serial.println("MPU6050 initialized");
}

void initLEDs() {
    Serial.println("Initializing LEDs...");
    FastLED.addLeds<WS2812B, LED_PIN, GRB>(leds,
NUM_LEDS);
    FastLED.setBrightness(BRIGHTNESS);
    FastLED.clear(true);
    Serial.println("LEDs initialized");
}

void initParticles() {
    Serial.println("Initializing particles...");
    int index = 0;

    for (int y = MATRIX_HEIGHT - 4; y <
MATRIX_HEIGHT; y++) {
        for (int x = 0; x < MATRIX_WIDTH && index <
FLUID_PARTICLES; x++) {
            particles[index].position =
{static_cast<float>(x), static_cast<float>(y)};

```

```

        particles[index].velocity = {0.0f,
0.0f};
        index++;
    }
}

    Serial.printf("Total particles initialized:
%d\n", index);
}

void MPUTask(void *parameter) {
    while (true) {
        int16_t ax, ay, az;
        mpu.getAcceleration(&ax, &ay, &az);

        portENTER_CRITICAL(&dataMux);
        acceleration.x = -constrain(ax / 16384.0f,
-1.0f, 1.0f);
        acceleration.y = constrain(ay / 16384.0f, -
1.0f, 1.0f);
        portEXIT_CRITICAL(&dataMux);

        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

void LEDTask(void *parameter) {
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency =
pdMS_TO_TICKS(16);

    while (true) {
        checkButton();
        updateParticles();
    }
}

```

```
        drawParticles();
        vTaskDelayUntil(&xLastWakeTime,
xFrequency);
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("Starting initialization...");

    // Initialize button pin
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    Wire.begin(SDA_PIN, SCL_PIN);
    Wire.setClock(400000);

    initMPU6050();
    initLEDs();
    initParticles();

    xTaskCreatePinnedToCore(
        MPUTask,
        "MPUTask",
        4096,
        NULL,
        2,
        NULL,
        0
    );

    xTaskCreatePinnedToCore(
        LEDTask,
        "LEDTask",
```

```

    4096,
    NULL,
    1,
    NULL,
    1
);

Serial.println("Setup complete");
}

void loop() {
    vTaskDelete(NULL);
}

```

